



RC

# Rocket.Chat

## Full System Documentation

Setup | Architecture | Plugin Development | Translation Integration

---

UgaJapa Connect Project | Translation Group

Benard Emuduko | Makerere University | June 2026

```
$ git clone https://github.com/RocketChat/Rocket.Chat.git
$ cd Rocket.Chat && yarn && yarn dsv
```

# 1. What is Rocket.Chat?

Rocket.Chat is a free, open-source, self-hosted team communication platform built with TypeScript and Node.js on a Meteor framework, using MongoDB as the primary database. First released in 2015, it has grown to 42,000+ GitHub stars and is used by organisations including Deutsche Telekom, Lufthansa, the US Navy, and the European Central Bank. The latest stable version is **8.4.0** (May 2026).

For the UgaJapa Connect project, Rocket.Chat serves as the **base platform** on top of which the translation module (EN/Luganda to/from Japanese with cultural intelligence hints) will be built as a Rocket.Chat App using the Apps Engine framework.

| Property            | Value                                                                                                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GitHub Repository   | <a href="https://github.com/RocketChat/Rocket.Chat">https://github.com/RocketChat/Rocket.Chat</a>                                                                     |
| Apps Engine Repo    | <a href="https://github.com/RocketChat/Rocket.Chat/tree/develop/packages/apps-engine">https://github.com/RocketChat/Rocket.Chat/tree/develop/packages/apps-engine</a> |
| License             | MIT (Community Edition) — free and open source                                                                                                                        |
| Backend Language    | TypeScript / Node.js (Meteor framework)                                                                                                                               |
| Frontend Language   | React + TypeScript                                                                                                                                                    |
| Database            | MongoDB                                                                                                                                                               |
| Plugin Language     | TypeScript (Apps Engine)                                                                                                                                              |
| Latest Version      | 8.4.0 (May 2026)                                                                                                                                                      |
| Default Port        | 3000 (HTTP)                                                                                                                                                           |
| Mobile Apps         | iOS + Android (React Native — separate repo)                                                                                                                          |
| Apps Engine Version | @rocket.chat/apps-engine v1.59.1                                                                                                                                      |

## 2. Getting the Code — Clone & Run

### 2.1 Prerequisites

| Tool    | Version Required                      | Install Command                                                                 |
|---------|---------------------------------------|---------------------------------------------------------------------------------|
| Node.js | Check package.json engines.node field | nvm install                                                                     |
| nvm     | 0.39.2 or later                       | curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.2/install.sh   bash |
| Yarn    | 1.x                                   | npm install -g yarn                                                             |
| Git     | 2.x                                   | sudo apt install git (Ubuntu)                                                   |
| MongoDB | 6.0+ (or use Docker)                  | See Section 3.2                                                                 |
| Deno    | Required for some scripts             | curl -fsSL https://deno.land/x/install/install.sh   sh                          |
| VS Code | Recommended IDE                       | code.visualstudio.com                                                           |

### 2.2 Clone the Main Rocket.Chat Repository

```
# Step 1: Fork on GitHub first (go to github.com/RocketChat/Rocket.Chat and click Fork) #
Then clone YOUR fork: git clone https://github.com/YOUR-USERNAME/Rocket.Chat.git cd
Rocket.Chat # Add upstream remote to stay in sync git remote add upstream
https://github.com/RocketChat/Rocket.Chat.git # Verify remotes git remote -v # origin
https://github.com/YOUR-USERNAME/Rocket.Chat.git # upstream
https://github.com/RocketChat/Rocket.Chat.git
```

### 2.3 Install Dependencies & Run Dev Server

```
# Find the required Node.js version cat package.json | grep -A4 engines | grep node #
Install that exact version with nvm nvm install <version-from-above> nvm use
<version-from-above> # Install all dependencies yarn # Start the development server (runs on
http://localhost:3000) yarn dsv # When you see this, the server is ready: #
+-----+ # | SERVER RUNNING | # | Rocket.Chat
Version: 8.x.x | # | NodeJS Version: xx.x.x | # | MongoDB Version: 6.x.x | # | MongoDB
Engine: wiredTiger | # | Platform: linux | #
+-----+ # Access at: http://localhost:3000
```

#### First Run

The first time yarn dsv runs, it downloads Meteor dependencies which can take 10-20 minutes. Subsequent starts are much faster. Create your admin account at <http://localhost:3000> when prompted.

### 2.4 Quick Docker Setup (Recommended for Testing)

If you just want a running Rocket.Chat to test your plugin against, Docker is the fastest path:

```
# Pull and run Rocket.Chat with MongoDB in one command docker run --name rocketchat -p
3000:3000 \ --link mongo_rocketchat \ -e
```

```
MONGO_URL=mongodb://mongo_rocketchat:27017/rocketchat \ -e ROOT_URL=http://localhost:3000 \
-e PORT=3000 \ -d rocketchat/rocket.chat:latest # Or use the official docker-compose: git
clone https://github.com/RocketChat/Docker.Official.Image.git cd Docker.Official.Image cp
env.example .env docker compose up -d # Access at http://localhost:3000
```

### 3. Rocket.Chat System Architecture

#### 3.1 High-Level Architecture

Rocket.Chat uses a monorepo structure containing the server, web client, and supporting packages. The core technology stack is:

| Layer              | Technology             | Role                                                                        |
|--------------------|------------------------|-----------------------------------------------------------------------------|
| Framework          | Meteor (Node.js)       | Full-stack reactive framework; handles real-time data sync via DDP protocol |
| Backend Runtime    | Node.js + TypeScript   | Server-side business logic, REST API, WebSocket hub                         |
| Frontend           | React + TypeScript     | Web client UI — channels, messages, settings, admin panel                   |
| Database           | MongoDB                | Primary data store — messages, users, rooms, settings                       |
| Real-time          | WebSocket (DDP)        | Meteor's Distributed Data Protocol for live message updates                 |
| REST API           | Express.js             | HTTP REST endpoints for external integrations and apps                      |
| Plugin Runtime     | Apps Engine            | Sandboxed TypeScript runtime for Rocket.Chat Apps                           |
| Search             | MongoDB text / Elastic | Full-text message and user search                                           |
| File Storage       | GridFS / S3 / local    | Message attachments, avatars, exports                                       |
| Cache              | In-memory / Redis      | Session caching, rate limiting, cluster coordination                        |
| Push Notifications | APNs / FCM via gateway | iOS and Android push via Rocket.Chat's push gateway or custom               |

#### 3.2 Monorepo Structure

```
Rocket.Chat/ # Root monorepo apps/ meteor/ # Main Rocket.Chat server (Meteor) client/ #
React frontend components server/ # Server-side code app/ # Feature modules (channels,
messages, users...) definition/ # TypeScript type definitions packages/ apps-engine/ #
Plugin runtime (THE KEY FOR OUR WORK) core-typings/ # Shared TypeScript types fuselage/ #
Rocket.Chat UI component library rest-typings/ # REST API TypeScript types i18n/ #
Internationalisation strings ui-kit/ # UI component blocks .docker/ # Docker configuration
package.json # Root workspace config yarn.lock # Locked dependencies
```

#### 3.3 Message Flow (How a Message Gets Delivered)

Understanding this flow is critical for building the translation plugin, because the plugin intercepts messages at Step 4:

| St ep | Actor                | What Happens                                                                |
|-------|----------------------|-----------------------------------------------------------------------------|
| 1     | Client (browser/app) | User types a message and presses Enter. React component calls sendMessage() |

| Step | Actor                  | What Happens                                                                                    |
|------|------------------------|-------------------------------------------------------------------------------------------------|
| 2    | Meteor DDP             | Message is sent via WebSocket (DDP) to the server as a method call                              |
| 3    | Server — Meteor Method | sendMessage() method is called on the server; basic validation performed                        |
| 4    | Apps Engine Hook       | beforeSendMessage hook fires — OUR TRANSLATION PLUGIN runs here, modifying the message          |
| 5    | Server — DB Write      | Modified message (with translation + hints attached) is saved to MongoDB                        |
| 6    | Publication            | Meteor reactivity publishes the new message to all subscribers of that room                     |
| 7    | Client — Render        | All connected clients receive the message via WebSocket and React re-renders the UI             |
| 8    | Our UI Component       | The client-side UIKit component reads translation data from message.customFields and renders it |
| 9    | Push (if offline)      | If recipients are offline, FCM/APNs push notification is sent via push gateway                  |

## 4.1 MongoDB Collections

## 4.2 How Translation Data is Stored in Messages

Page 7

## 5. Apps Engine — Plugin System Deep Dive

The Apps Engine is the sandboxed plugin runtime that allows developers to extend Rocket.Chat without modifying its core code. Apps are written in TypeScript, packaged with the rc-apps CLI, and deployed to a running Rocket.Chat instance. This is where ALL of our translation work happens.

### 5.1 Install the Apps Engine CLI

```
# Install the Rocket.Chat Apps CLI globally npm install -g @rocket.chat/apps-cli # Verify
installation rc-apps --version # Expected: @rocket.chat/apps-cli/x.x.x # Create a new app
project rc-apps create # Follow prompts: # App Name: UgaJapa Translation App # App
Description: Real-time EN/Luganda<->Japanese translation with cultural hints # Author Name:
UgaJapa Connect Team # Author Email: team@ugajapa.ac.ug # Author Homepage:
https://github.com/ugajapa-connect
```

### 5.2 Generated App Structure

```
ugajapa-translation-app/ UgaJapaTranslationApp.ts # Main app class (entry point) app.json #
App manifest package.json # Dependencies tsconfig.json # TypeScript config .rcappsconfig #
CLI deploy config # We will add these: handlers/ TranslationHandler.ts # Message
interception hook services/ ClaudeTranslationService.ts # Claude API calls
CulturalHintsService.ts # Cultural intelligence logic LanguageDetectionService.ts #
Auto-detect message language ui/ TranslationBlock.ts # UIKit block for rendering translation
CulturalHintBlock.ts # UIKit block for cultural alerts commands/ SetLanguageCommand.ts #
/set-language slash command TranslateCommand.ts # /translate slash command settings/
AppSettings.ts # Admin-configurable settings i18n/ en.json # English strings ja.json #
Japanese strings
```

### 5.3 Key Apps Engine Hooks Available

| Hook                     | When It Fires                                        | Our Use                                                        |
|--------------------------|------------------------------------------------------|----------------------------------------------------------------|
| IPreMessageSentPrevent   | Before a message is saved — can BLOCK the send       | Block messages with harmful content (optional safety check)    |
| IPreMessageSentExtend    | Before a message is saved — can ADD data to it       | PRIMARY HOOK: Add translation + cultural hints to customFields |
| IPreMessageSentModify    | Before a message is saved — can CHANGE message text  | Optionally modify message text (we prefer to keep original)    |
| IPostMessageSent         | After a message is saved — cannot change the message | Log translation events, trigger analytics                      |
| IPreRoomUserJoined       | Before a user joins a room                           | Send welcome message with language setup instructions          |
| IPostRoomUserJoined      | After a user joins a room                            | Auto-configure language preference for new room members        |
| IUIKitInteractionHandler | When user clicks a button in a UIKit block           | Handle "Show Original" / "Retranslate" button clicks           |
| ISlashCommandPreviewer   | Slash command preview generation                     | Show translation preview before sending /translate command     |





## 6. Building the UgaJapa Translation Plugin

### 6.1 app.json — App Manifest

```
{ "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890", "version": "1.0.0", "requiredApiVersion":
"^1.4.0", "iconFile": "icon.png", "author": { "name": "UgaJapa Connect Team", "homepage":
"https://github.com/ugajapa-connect", "support": "team@ugajapa.ac.ug" }, "name": "UgaJapa
Translation", "nameSlug": "ugajapa-translation", "classFile": "UgaJapaTranslationApp.ts",
"description": "Real-time EN/Luganda to Japanese translation with cultural intelligence
hints", "categories": ["Productivity", "Communication"], "permissions": [ { "name":
"networking" }, { "name": "message.read" }, { "name": "message.write" }, { "name":
"room-read" }, { "name": "user-read" }, { "name": "env-settings-read" } ] }
```

### 6.2 UgaJapaTranslationApp.ts — Main App Class

```
import { App } from "@rocket.chat/apps-engine/definition/App"; import { IAppInfo } from
"@rocket.chat/apps-engine/definition/metadata"; import { ILogger } from
"@rocket.chat/apps-engine/definition/accessors"; import { IAppAccessors } from
"@rocket.chat/apps-engine/definition/accessors"; import { TranslationHandler } from
"./handlers/TranslationHandler"; import { SetLanguageCommand } from
"./commands/SetLanguageCommand"; import { TranslateCommand } from
"./commands/TranslateCommand"; import { AppSettings } from "./settings/AppSettings"; export
class UgaJapaTranslationApp extends App { constructor(info: IAppInfo, logger: ILogger,
accessors: IAppAccessors) { super(info, logger, accessors); } // Register all hooks and
commands when app is enabled public async extendConfiguration(config: any): Promise<void> {
// Message translation hook config.slashCommands.provideSlashCommand(new
SetLanguageCommand(this)); config.slashCommands.provideSlashCommand(new
TranslateCommand(this)); } // Called when app is installed or updated public async
onEnable(): Promise<boolean> { this.getLogger().debug("UgaJapa Translation App enabled!");
return true; } // App settings definition public async getSettings(): Promise<Array<any>> {
return AppSettings; } }
```

### 6.3 TranslationHandler.ts — The Core Hook

```
import { IPreMessageSentExtend, IModifyExtender, IRead, IHttp } from
"@rocket.chat/apps-engine/definition/accessors"; import { IMessage } from
"@rocket.chat/apps-engine/definition/messages"; import { ClaudeTranslationService } from
"./services/ClaudeTranslationService"; import { LanguageDetectionService } from
"./services/LanguageDetectionService"; export class TranslationHandler implements
IPreMessageSentExtend { // This fires for EVERY message before it is saved to the database
public async executePreMessageSentExtend( message: IMessage, extend: IModifyExtender, read:
IRead, http: IHttp ): Promise<IMessage> { // Skip system messages, empty messages, or
already-translated messages if (!message.text || message.customFields?.ugajapa_translation)
{ return message; } // Determine source and target language const sourceLang = await
LanguageDetectionService.detect(message.text); const room = await
read.getRoomReader().getById(message.room.id); const targetLang =
(room?.customFields?.ugajapa_targetLang as string) || "ja"; // Skip if source and target are
same if (sourceLang === targetLang) return message; // Call Claude API for translation +
cultural hints const apiKey = await read.getEnvironmentReader()
.getSettings().getValueById("ugajapa_claude_api_key"); const result = await
ClaudeTranslationService.translate({ text: message.text, sourceLang, targetLang, apiKey,
http }); // Attach translation data to message customFields const extendMsg =
extend.extendMessage(); extendMsg.addCustomField("ugajapa_translation", { original:
message.text, translated: result.translation, sourceLang, targetLang, culturalHint:
result.culturalHint || null }); return extendMsg.getMessage(); } }
```

## 6.4 ClaudeTranslationService to Claude API Integration

```
import { IHttp } from "@rocket.chat/apps-engine/definition/accessors"; interface
TranslationRequest { text: string; sourceLang: string; // "en" | "lg" (Luganda) | "ja"
targetLang: string; apiKey: string; http: IHttp; } interface TranslationResult {
translation: string; culturalHint: string | null; } export class ClaudeTranslationService {
static async translate(req: TranslationRequest): Promise<TranslationResult> { const
langNames: Record<string, string> = { "en": "English", "lg": "Luganda", "ja": "Japanese" };
const prompt = `You are a professional translator specialising in Uganda-Japan
communication. Translate the following ${langNames[req.sourceLang]} text to
${langNames[req.targetLang]}. Also check if the message contains any cultural nuance that
could cause misunderstanding between Ugandan and Japanese communicators (e.g. indirect
refusals, honorific implications, directness vs indirectness, silence interpretation).
Respond in this exact JSON format only: { "translation": "<translated text>",
"culturalHint": "<hint text or null>" } Text to translate: "${req.text}"`; const response =
await req.http.post( "https://api.anthropic.com/v1/messages", { headers: { "x-api-key":
req.apiKey, "anthropic-version": "2023-06-01", "content-type": "application/json" }, data: {
model: "claude-sonnet-4-6", max_tokens: 1024, messages: [{ role: "user", content: prompt }]
} } ); const raw = response.data.content[0].text; const parsed =
JSON.parse(raw.replace(/``json|``/g, "").trim()); return { translation:
parsed.translation, culturalHint: parsed.culturalHint || null }; }
```

## 6.5 AppSettings to Admin Configuration

```
import { ISetting, SettingType } from "@rocket.chat/apps-engine/definition/settings"; export
const AppSettings: ISetting[] = [ { id: "ugajapa_claude_api_key", type:
SettingType.PASSWORD, packageValue: "", required: true, public: false, i18nLabel: "Claude
API Key", i18nDescription: "Your Anthropic Claude API key for translations" }, { id:
"ugajapa_default_target_lang", type: SettingType.SELECT, packageValue: "ja", values: [ {
key: "ja", i18nLabel: "Japanese" }, { key: "en", i18nLabel: "English" }, { key: "lg",
i18nLabel: "Luganda" } ], required: true, public: true, i18nLabel: "Default Target Language"
}, { id: "ugajapa_show_hints", type: SettingType.BOOLEAN, packageValue: true, required:
false, public: true, i18nLabel: "Show Cultural Intelligence Hints" }, { id:
"ugajapa_auto_translate", type: SettingType.BOOLEAN, packageValue: true, required: false,
public: true, i18nLabel: "Auto-translate all messages" } ];
```

## 6.6 SetLanguageCommand to Slash Command

```
import { ISlashCommand, SlashCommandContext } from
"@rocket.chat/apps-engine/definition/slashcommands"; import { IModify, IRead } from
"@rocket.chat/apps-engine/definition/accessors"; export class SetLanguageCommand implements
ISlashCommand { public command = "set-language"; public i18nParamsExample = "ja | en | lg";
public i18nDescription = "Set translation target language for this channel"; public
providesPreview = false; public async executor( context: SlashCommandContext, read: IRead,
modify: IModify ): Promise<void> { const lang = context.getArguments()[0]?.toLowerCase();
const validLangs = ["en", "lg", "ja"]; if (!validLangs.includes(lang)) { await
this.sendMsg(context, modify, "Invalid language. Use: /set-language ja | en | lg"); return;
} // Store language preference on the room const roomUpdater = modify.getExtender()
.extendRoom(context.getRoom().id, context.getSender()); await
roomUpdater.addCustomField("ugajapa_targetLang", lang); await roomUpdater.finish(); const
langNames: Record<string, string> = { "en": "English", "lg": "Luganda", "ja": "Japanese" };
await this.sendMsg(context, modify, `Translation target set to *${langNames[lang]}* for this
channel.`); } private async sendMsg(ctx: SlashCommandContext, modify: IModify, text:
string): Promise<void> { const builder = modify.getCreator()
.startMessage().setText(text).setRoom(ctx.getRoom()); await
modify.getCreator().finish(builder); } }
```

## 7. Deploying & Testing the Plugin

### 7.1 Deploy to Local Dev Server

```
# Navigate into your app folder cd ugajapa-translation-app # Deploy to your local
Rocket.Chat instance rc-apps deploy --url http://localhost:3000 \ --username admin \
--password your-admin-password # For HTTPS servers: rc-apps deploy --url
https://your-rocketchat.com \ --username admin \ --password your-admin-password # Watch mode
(auto-redeploy on file changes during development) rc-apps watch --url http://localhost:3000
\ --username admin \ --password your-admin-password
```

### 7.2 Configure the Plugin in Rocket.Chat Admin

| St<br>ep | Where                                  | Action                                                                  |
|----------|----------------------------------------|-------------------------------------------------------------------------|
| 1        | Admin Panel > Apps                     | Find "UgaJapa Translation" in the installed apps list                   |
| 2        | App > Settings tab                     | Paste your Claude API key into the "Claude API Key" field               |
| 3        | App > Settings tab                     | Set default target language (Japanese for Uganda-Japan project)         |
| 4        | App > Settings tab                     | Enable "Show Cultural Intelligence Hints" toggle                        |
| 5        | Any channel                            | Type /set-language ja to set that channel's target language to Japanese |
| 6        | Any channel                            | Send a test message in English: "Hello, how are you?"                   |
| 7        | Message appears with translation below | Verify Japanese translation appears under the original message          |
| 8        | Admin Panel > Apps > Logs              | Check app logs for errors if translation does not appear                |

### 7.2 Testing the Translation Flow

```
# Test 1: English to Japanese # Send in channel: "The meeting is confirmed for tomorrow." #
Expected: Japanese translation appears below message # Expected: No cultural hint
(straightforward statement) # Test 2: Cultural hint trigger # Send in channel: "That might
be a bit difficult..." # Expected: Japanese translation appears # Expected: Cultural hint
appears: "In Japanese communication, this phrase # often indicates an indirect refusal..." #
Test 3: Luganda to Japanese # First set language: /set-language lg # Send: "Webale nyo,
nkola bulungi" # Expected: Japanese translation appears # Test 4: Slash command test #
/set-language ja => sets channel target to Japanese # /set-language en => sets channel
target to English # /set-language xyz => shows error message
```

## 8. Feature Interaction Map

This section describes how all features of the UgaJapa Connect translation plugin interact with Rocket.Chat's built-in features.

| Rocket.Chat Feature | Our Plugin Interaction                          | How It Works                                                                                                               |
|---------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Channels            | Language settings stored per channel            | /set-language stores target lang in channel.customFields; all messages in that channel are translated to that language     |
| Direct Messages     | Translation works in DMs too                    | Same IPreMessageSentExtend hook fires in DMs — Ugandan user DMing Japanese supervisor gets auto-translation                |
| Message threads     | Thread replies are also translated              | Hook fires on all messages regardless of whether they are thread replies; translation attaches to each reply independently |
| Message reactions   | No modification — reactions work normally       | Emoji reactions are not messages; they do not trigger our translation hook                                                 |
| File uploads        | File messages are skipped by default            | Messages containing only file attachments (no text) are skipped by the hook to avoid empty translation calls               |
| Slash commands      | /set-language and /translate commands added     | Our app registers two new slash commands visible in the slash command autocomplete list                                    |
| User mentions       | @mentions preserved in translation              | Claude API preserves @username mentions in translated text (prompted to do so in system prompt)                            |
| Search              | Only original message text is indexed           | MongoDB text index uses msg field (original) — translation in customFields is not searched by default                      |
| Push notifications  | Original message sent as push                   | Push notification shows original text; translation visible when user opens the app in the channel                          |
| Admin panel         | Plugin settings in App Settings tab             | Claude API key, language defaults, and toggle for hints all configurable from Admin > Apps > UgaJapa Translation           |
| Room info sidebar   | Language badge showing current target lang      | UI component in room header shows which language is active (e.g. "Translating to: JA")                                     |
| Message actions     | "Retranslate" and "Show Original" actions added | Right-click a message to access our custom actions; triggers UIKit modal with retranslation options                        |

## 9. Rocket.Chat REST API Reference

The REST API is used for external integrations and automation. All endpoints are prefixed with /api/v1/. Authentication is via header: X-Auth-Token + X-User-Id.

### 9.1 Authentication

```
# Get auth token curl -X POST http://localhost:3000/api/v1/login \ -H "Content-Type: application/json" \ -d '{"user": "admin", "password": "adminpassword"}' # Response: # { "status": "success", "data": { # "userId": "abc123", # "authToken": "xyz789...", # "me": { "username": "admin", ... } # } # } # Use in all subsequent requests: curl http://localhost:3000/api/v1/me \ -H "X-Auth-Token: xyz789..." \ -H "X-User-Id: abc123"
```

### 9.2 Key API Endpoints

| Method | Endpoint                         | Description                                           |
|--------|----------------------------------|-------------------------------------------------------|
| POST   | /api/v1/login                    | Authenticate and get auth token                       |
| GET    | /api/v1/me                       | Get current user info                                 |
| GET    | /api/v1/channels.list            | List all public channels                              |
| GET    | /api/v1/channels.history         | Get message history for a channel                     |
| POST   | /api/v1/channels.create          | Create a new public channel                           |
| POST   | /api/v1/chat.sendMessage         | Send a message to a channel                           |
| POST   | /api/v1/chat.update              | Update (edit) an existing message                     |
| POST   | /api/v1/chat.delete              | Delete a message                                      |
| GET    | /api/v1/chat.getMessage          | Get a single message by ID                            |
| GET    | /api/v1/users.list               | List all users                                        |
| GET    | /api/v1/rooms.info               | Get room info including customFields                  |
| POST   | /api/v1/rooms.saveRoomSettings   | Update room settings including customFields           |
| GET    | /api/v1/apps                     | List installed apps (admin only)                      |
| GET    | /api/v1/apps/{id}/settings       | Get app settings                                      |
| POST   | /api/v1/apps/{id}/settings       | Update app settings                                   |
| POST   | /api/v1/channels.setCustomFields | Set custom fields on a channel (for language setting) |

# 10. Security Configuration

## 10.1 Essential Security Settings

| Setting           | Location                        | Recommended Value                                                |
|-------------------|---------------------------------|------------------------------------------------------------------|
| HTTPS             | NGINX reverse proxy             | Always use HTTPS in production — never expose port 3000 directly |
| 2FA               | Admin > Settings > Accounts     | Enable for all users; enforce for admin accounts                 |
| Registration      | Admin > Settings > Accounts     | Set to "Manual Approval" or "Invite Only" for private instances  |
| Password policy   | Admin > Settings > Accounts     | Minimum 8 chars, require complexity                              |
| Claude API Key    | App Settings (encrypted)        | Stored as PASSWORD type — never visible in UI after saving       |
| Rate limiting     | Admin > Settings > Rate Limiter | Enable to prevent API abuse                                      |
| File upload types | Admin > Settings > File Upload  | Restrict to safe types: image, pdf, text                         |
| Session expiry    | Admin > Settings > Accounts     | 7 days for web, 30 days for mobile                               |
| Audit log         | Admin > Reports > Audit Log     | Enable to track admin actions (Enterprise)                       |
| IP allowlist      | NGINX config                    | Restrict admin access to known IPs                               |

## 10.2 Production NGINX Configuration

```
server { listen 443 ssl http2; server_name chat.ugajapa.ac.ug; ssl_certificate
/etc/letsencrypt/live/chat.ugajapa.ac.ug/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/chat.ugajapa.ac.ug/privkey.pem; ssl_protocols TLSv1.2 TLSv1.3; #
WebSocket support (required for real-time messaging) location ~ /sockjs/|/websocket {
proxy_pass http://localhost:3000; proxy_http_version 1.1; proxy_set_header Upgrade
$http_upgrade; proxy_set_header Connection "upgrade"; proxy_read_timeout 86400; } location /
{ proxy_pass http://localhost:3000; proxy_set_header Host $host; proxy_set_header X-Real-IP
$remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header
X-Forwarded-Proto https; } } # Redirect HTTP to HTTPS server { listen 80; server_name
chat.ugajapa.ac.ug; return 301 https://$host$request_uri; }
```



## 11. Quick Reference

### 11.1 All Clone Commands

```
# === MAIN ROCKET.CHAT SERVER === git clone https://github.com/RocketChat/Rocket.Chat.git cd
Rocket.Chat yarn && yarn dsv # Access: http://localhost:3000 # === MOBILE APP (React Native)
=== git clone https://github.com/RocketChat/Rocket.Chat.ReactNative.git cd
Rocket.Chat.ReactNative yarn && yarn android # or yarn ios # === APPS ENGINE (plugin SDK)
=== git clone https://github.com/RocketChat/Rocket.Chat.git # Apps Engine is in:
packages/apps-engine/ # === FUSELAGE (UI component library) === git clone
https://github.com/RocketChat/fuselage.git # === CREATE OUR TRANSLATION PLUGIN === npm
install -g @rocket.chat/apps-cli rc-apps create # follow prompts cd ugajapa-translation-app
rc-apps deploy --url http://localhost:3000 --username admin --password pass # === DEPLOY IN
WATCH MODE (auto-redeploy) === rc-apps watch --url http://localhost:3000 --username admin
--password pass
```

### 11.2 Key Admin URLs

| URL                                           | Purpose                      |
|-----------------------------------------------|------------------------------|
| http://localhost:3000                         | Main chat interface          |
| http://localhost:3000/admin                   | Admin panel home             |
| http://localhost:3000/admin/apps              | Installed apps management    |
| http://localhost:3000/admin/settings          | Server settings              |
| http://localhost:3000/admin/users             | User management              |
| http://localhost:3000/admin/rooms             | Room/channel management      |
| http://localhost:3000/admin/reports           | Audit logs and reports       |
| http://localhost:3000/api/v1/info             | Server info + version (JSON) |
| http://localhost:3000/api/v1/chat.sendMessage | Send message via REST API    |

### 11.2 Useful rc-apps Commands

```
rc-apps create # Scaffold a new app rc-apps deploy # Deploy app to server rc-apps watch #
Deploy + auto-watch for changes rc-apps package # Build .zip package for distribution
rc-apps validate # Validate app.json and TypeScript rc-apps info <url> # Get server info
rc-apps --help # Show all commands npx rc-apps@latest create # Always use latest CLI version
```

*This documentation was prepared by Benard Emuduko for the UgaJapa Connect project at Makerere University.*

*Official Rocket.Chat developer documentation: [developer.rocket.chat](https://developer.rocket.chat). Official GitHub:*

*[github.com/RocketChat/Rocket.Chat](https://github.com/RocketChat/Rocket.Chat). Apps Engine npm: [@rocket.chat/apps-cli](https://www.npmjs.com/package/@rocket.chat/apps-cli).*